

Experimentelle Analyse von Algorithmen für fair-k-center mit Ausreißern

Bachelorarbeit

von

Carsten Krollmann

aus

Pirmasens

vorgelegt bei

Lehrstuhl für Algorithmen und Datenstrukturen

Prof. Dr. Melanie Schmidt

Heinrich-Heine-Universität Düsseldorf

Juni 2024

Betreuer:

Dr. Daniel Schmidt

Zusammenfassung

Wenn Algorithmen neu entwickelt und theoretische Eigenschaften gezeigt werden, fehlt häufig das Wissen darüber, wie diese Algorithmen auf echten Daten performen. Dies muss dann in der Regel über Experimente auf echten Daten überprüft werden.

Diese Bachelorarbeit beschäftigt sich mit der experimentellen Analyse von Fair-Clustering Algorithmen. Genauer um Algorithmen, welche nicht nur versuchen möglichst kleine Cluster zu erzeugen, sondern zusätzlich ein gewähltes, faires Attribut gleichmäßig auf die Cluster zu verteilen. Dabei soll herausgefunden werden, wie gut die von den Algorithmen produzierte Lösung im Vergleich zueinander ist. Außerdem soll auch die Laufzeit verglichen werden. Explizit geht es um folgende zwei Algorithmen: Der Red-Clustering-Algorithmus mit einer bewiesenen Approximationsgüte von $\mathcal{O}(n^4)$ und dem Fast-Anchor-Algorithmus mit einer Approximationsgüte von $\mathcal{O}(n^3)$

Um die Algorithmen gegeneinander zu testen wurden sie zuerst in C++ implementiert und dann automatisch, mit Hilfe von Python-Skripts, getestet. Dazu wurden drei verschiedene Datensätze verwendet, wobei diese in einzelne kleine Samples zerlegt wurden.

Die Tests zeigen, dass der Red-Clustering Algorithmus, im Schnitt die bessere Lösung, also eine Lösung mit geringeren Kosten, berechnet. Zusätzlich ist der Red-Clustering-Algorithmus deutlich schneller als der verglichene Fast-Anchor-Algorithmus. Beim Testen sind einige Randfälle aufgetreten, wodurch das Clustering des Fast-Anchor-Algorithmus eine schlechtere Lösung produziert. Es ist ebenfalls aufgefallen, dass mit steigender Dimensionalität der Daten der Abstand der Kosten zwischen den Algorithmen zunimmt.

In dieser Arbeit wurden nur Algorithmen verglichen, welche eine 1:1 Verteilung auf den fairen Attributen suchen. Jedoch lassen sich beide Algorithmen auch für beliebige Verhältnisse und für mehr Farben verallgemeinern.

Inhaltsverzeichnis

List of Algorithms	v
Abbildungsverzeichnis	vi
1 Einleitung	1
1.1 Motivation	1
1.2 Aufbau der Arbeit	2
2 Algorithmen	3
2.1 Matching	3
2.1.1 Bipatites Matching	3
2.2 Flussnetzwerke	4
2.2.1 Flusseigenschaften	4
2.2.2 Max-Flow Algorithmus	4
2.2.3 Min-Cost-Flow	5
2.3 Gonzalez	5
2.4 Fairletts	5
2.5 Fair-Clustering mit Ausreißer	7
2.5.1 Ausreißer-Punkte	7
2.5.2 Red-Clustering	7
2.5.3 Fast-Anchor-Clustering	10
3 Aufbereiten der Daten	15
3.1 Ursprungsdaten	15
3.1.1 Problem der Datendokumentation	16
3.2 Cleaning-Pipeline	16
3.2.1 Normalisierung	16
3.2.2 Reinigen und Filtern	16

3.2.3	Subsampling	17
3.2.4	Format der Datensätze	17
4	Experimente	18
4.1	Implementierung der Algorithmen	18
4.2	Automatisierte Analyse	19
4.3	Testumgebung	19
4.4	Graphen	20
4.4.1	Güte-Analyse	20
4.4.2	Laufzeit-Analyse	22
5	Auswertung	26
5.1	Güte der Algorithmen	26
5.1.1	Center-Aware Problem	27
5.1.2	Tests mit Min-Cost-Flow	27
5.2	Laufzeitanalyse	28
5.3	Mögliche Verbesserungen	28
5.3.1	Laufzeit	29
5.3.2	Kosten	29
5.3.3	Analyse	30
5.4	Fazit	30
	Literatur	31

List of Algorithms

1	Gonzalez-Algorithmus	6
2	Farilett-Finder Red Clustering	9
3	Red Clustering Algorithmus	10
4	Farilett-Finder Fast-Anchor Clustering	12
5	Gonzalez-Algorithmus mit Fairletts	13
6	Fast-Anchor Clustering Algorithmus	14

Abbildungsverzeichnis

4.1	Zusammengefasste Kosten der Cluster bis zu 30 Cluster	21
4.2	Kosten der Cluster mit Median und Min/Max des Zensusdatensatzes mit den ersten 10 Subsamples	22
4.3	Unterschied in den Radiuskosten der Cluster zwischen Fairlettbildung mit einfa- chen Max-Flow und Min-Cost-Flow bei Fast-Anchor-Clustering	23
4.4	Laufzeit des Gonzalez-Algorithmus	24
4.5	Laufzeit des Red-Clustering-Algorithmus	24
4.6	Laufzeit des Fast-Anchor-Algorithmus	25
5.1	Fast-Anchor-Clustering Edge-Case mit $k = 3$	28
5.2	Fast-Anchor-Clustering Edge-Case mit $k = 4$	28

Kapitel 1

Einleitung

1.1 Motivation

Große Mengen an Daten zu analysieren ist inzwischen ein alltägliches Problem. Vor allem die Mustererkennung und das Finden von Beziehungen zwischen Elementen aus den Daten ist von großer Bedeutung. Eine Möglichkeit solche Beziehungen zu finden ist das Clustering. Dabei versucht man algorithmisch „nahe“ Punkte zu Gruppen, sogenannten Clustern, zusammenzufassen. Die Nähe kann hier sehr unterschiedlich definiert sein. Diese Art der Datenverarbeitung ist eine Form des unüberwachten maschinelle Lernens, da wir versuchen eine Eigenschaft in den Daten zu finden, ohne vorher Wissen über diese Eigenschaft zu besitzen.

Es zeigt sich jedoch, dass dieses Gruppieren in der Regel sehr schwer zu berechnen ist. Genauer gesagt sind die meisten Clustering-Probleme NP-Vollständig [Sch23b]. Man greift daher auf Algorithmen zurück, welche zwar schneller rechnen, aber dafür nicht die optimale Lösung liefern. Es gibt bereits Algorithmen, welche schon effizient und schnell diese nicht-optimalen Cluster berechnen, wie beispielsweise der Algorithmus von Gonzalez [Gon85], welcher mit einer 2er Approximation sehr gute Lösungen in polynomieller Zeit erzielen kann. Ein weiterer bekannter Algorithmus, welcher sehr viel in der Data Science benutzt wird, ist der k-means Algorithmus [Mac+67]. Dieser produziert vorallem in der Praxis sehr gute Lösungen.

Umso schwerer werden die Berechnungen, wenn man neben der „Nähe“ weitere Nebenbedingung an die berechneten Cluster stellt. Darunter fällt z.B. die gerechte Aufteilung auf Cluster. Dabei hat jeder Punkt eine zusätzliche Eigenschaft, welche fair auf die Cluster verteilt werden soll. Die oben genannt Algorithmen sind nicht in der Lage eine solche Nebenbedingung zu berücksichtigen. In dieser Arbeit wird abstrakt von Farben als faires Attribut gesprochen, doch diese Eigenschaft kann

beliebig gewählt werden. Beispiele dafür sind das Geschlecht, der Familienstand oder Ähnliches. Algorithmen welche dieses Clustering mit Nebenbedingung berechnen können, sind noch nicht gut erforscht. Daher werden wir in dieser Arbeit zwei Algorithmen, welche eine derartige Berechnung durchführen können, implementieren und ihre Güte, sowie ihre Laufzeit auf echten Daten auszuwerten.

1.2 Aufbau der Arbeit

In dieser Arbeit sollen verschiedene Clustering-Algorithmen getestet werden. Dazu stellen wir in Kapitel 2 erst einige grundlegende Algorithmen vor, welche verwendet werden. Größere Aufmerksamkeit bekommt dabei der Algorithmus von Gonzalez [Gon85], da dieser von den anderen Algorithmen als Unteroutine benutzt wird. Dann gehen wir auf die zu vergleichenden Algorithmen Red-Clustering [Chi+18] und Fast-Anchor-Clustering [Fas23] ein. Alle hier in dieser Arbeit verwendeten Algorithmen können nur Daten mit 2 Farben verarbeiten. Sie lassen sich allerdings auch verallgemeinern [Fas23] [Chi+18]. Im nächsten Kapitel 3 werden die Daten vorgestellt, mit welchen wir die Algorithmen testen wollen und wie wir sie für das Clustering vorbereiten.

Anschließend sprechen wir in Kapitel 4 darüber, auf welche Weise die Algorithmen getestet werden. Dabei wird einerseits auf die Testumgebung eingegangen und andererseits auch die Ergebnisse in Form einiger Graphen präsentiert. Zum Schluss 5 versuchen wir die Ergebnisse einzuordnen und schauen, welche Schlüsse wir daraus ziehen können. Sämtlicher Code für die Implementierung und Auswertung mit allen wichtigen Datensätzen sind in einem Repository hochgeladen [Kro]

Kapitel 2

Algorithmen

In der Arbeit werden verschiedene Algorithmen verwendet. Einige davon sind als bekannt vorausgesetzt und sollen nur kurz erläutert werden. Dannach gehen wir auf die implementierten Algorithmen ausführlicher ein.

2.1 Matching

Ein Matching ist eine Aufteilung eines Graphen in disjunkte 2er Paare. Die Paare sollen sich dabei nicht überschneiden und die Knoten eines Paares sind immer über eine Kante verbunden. Sind alle Punkte des Graphens in einem Paar spricht man von einem vollständigen Matching [Sch23a].

2.1.1 Bipatites Matching

Ein bipatiter Graph ist ein Graph, welcher in zwei disjunkte Mengen aufgeteilt werden kann, sodass nur Kanten zwischen den zwei Mengen existieren. Wird ein Matching in einem bipatiten Graphen gesucht spricht man von einem bipatiten Matching.

2.2 Flussnetzwerke

Die verwendeten Clustering-Algorithmen greifen auf die Berechnung eines maximalen Flusses in einem Graph-Netzwerk und das daraus resultierende Matching zurück. Dabei haben die Kanten zwischen den Knoten eine Kapazität, über welche ein Flusswert fließen kann. Zusätzlich gibt es noch zwei zusätzliche Knoten, die Quelle s und die Senke t [Sch23a].

2.2.1 Flusseigenschaften

Für einen Fluss, welcher durch ein Flussnetzwerk fließt, müssen immer folgende 3 Eigenschaften gelten:

1. Jede Kante im Netzwerk muss eine nicht-negative Kapazität und Fluss haben.
2. *Flusserhaltung*: Die Summe des einfließenden und des ausfließenden Flusses muss bei jedem Knoten mit Ausnahme von Quelle s und Senke t immer 0 sein. Bei der Quelle ist die Summe negativ, bei der Senke positiv.
3. *Kapazitätserhaltung*: Der Fluss welcher über eine Kante fließt, darf zu keinem Zeitpunkt die Kapazität dieser Kante überschreiten.

2.2.2 Max-Flow Algorithmus

Beim maximalen Fluss versucht man zwischen zwei Knoten, der Quelle s und der Senke t , möglichst viel Fluss zu schicken ohne die Flusseigenschaften zu verletzen [Sch23a]. Im Folgenden wird das benutzt, um Knoten verschiedener Farbe in einem Bipatiten Graphen zu matchen. Bei der hier verwendeten Implementierung wurde auf die Bibliothek Lemon Library [Lem] zurückgegriffen. Diese bietet die Möglichkeit selbst Graphen aufzubauen und dann den maximalen Fluss in diesen zu berechnen. Verwendet wurde hier der Preflow Algorithmus.

2.2.3 Min-Cost-Flow

Es ist möglich zusätzlich zum Flusswert noch weitere Eigenschaften zu berücksichtigen. Wenn die Kanten neben der Kapazität noch Kosten haben, dann ist es möglich einen Fluss zu berechnen, welcher einerseits maximal ist, aber andererseits möglichst wenig Kosten verursacht. Dabei sind die Kosten die Summe aller Kantenkosten der Kanten, über welche Fluss fließt. Hierbei wurde der Capacity Scaling Algorithms, ebenfalls aus der Lemon Library [Lem] verwendet.

2.3 Gonzalez

Der Algorithmus von Gonzalez [Gon85] berechnet auf einer Punktemenge ein Clustering mit dem k -center Problem als Zielfunktion. Allerdings berücksichtigt er dabei keine Fairness. Dennoch wird er als Teil der fairen Algorithmen verwendet. Der Algorithmus ist ein greedy-Algorithmus, welcher k Cluster sucht. Er hat also als Eingabe eine Menge an Punkten mit einer Metrik (dist) und einem k , welches die Anzahl an gewünschten Clustern angibt. Er beginnt damit, dass man einen beliebigen Punkt als Zentrum festlegt. Anschließend wird der Punkt, welcher am weitesten weg von diesem ersten Zentrum ist, als nächstes Zentrum gewählt. Für das dritte Zentrum wird der Punkt gewählt für den gilt, dass der Abstand zu den anderen Zentren maximal ist. Diese Schritte wiederholt man so oft bis, man k verschiedene Zentren hat. Zum Schluss wird jeder Punkt dem Zentrum zugewiesen, zu welchem er den geringsten Abstand hat. Zurück gibt der Algorithmus die Zentren, die Knoten mit ihrer Clusterzugehörigkeit und den maximalen Radius, also der größte Abstand von einem Knoten zu seinem Zentrum.

2.4 Fairletts

In dieser Arbeit soll faires Clustering analysiert werden, jedoch kann der Algorithmus von Gonzalez 1 eine solche Nebenbedingung nicht erfüllen. Wir benutzen hier sogenannte Fairletts. Fairletts sind die kleinstmögliche, faire Clustereinheit, welche die Nebenbedingung erfüllt. In dieser Analyse verwenden wir nur 1 : 1 Fairletts, die aus einem roten und einem blauen Punkt bestehen. Die Grundidee ist, dass wir immer, wenn wir einen Punkt zu einem Cluster zuweisen, zusätzlich sein Fairlett-Partner demselben Zentrum zugewiesen wird. So wird die Fairness nie verletzt [Fas23].

Algorithm 1 Gonzalez-Algorithmus

```

1: function GONZALEZ( $P$ ,  $dist$ ,  $k$ )
2:    $Z :=$  leere Zentrenmenge
3:    $d_{min} :=$  Distanz zu nächstem Zentrum
4:    $z_0$  beliebig aus  $P$ 
5:    $z_0.cluster = 0$ 
6:   for all  $x \in P$  do
7:      $d_{min} = dist(z_0, x)$ 
8:      $x.cluster = 0$ 
9:   end for
10:
11:   for  $i \in 0, \dots, k - 1$  do
12:      $z_i = P(argmax(d_{min}))$ 
13:      $z_i.cluster = i$ 
14:     for  $x \in P \setminus Z$  do
15:       if  $dist(z_i, x) < d_{min}[x]$  then
16:          $d_{min}[x] = dist(z_i, x)$ 
17:          $x.cluster = i$ 
18:       end if
19:     end for
20:   end for
21:    $r_{max} = max(d_{min})$ 
22:   return ( $Z, P, r_{max}$ )
23: end function

```

▷ in P wurden Punkte aktualisiert

In dieser Arbeit werden zwei Arten der Fairlettbildung betrachtet und sowohl ihre Laufzeit, als auch ihre Güte miteinander verglichen.

2.5 Fair-Clustering mit Ausreißer

2.5.1 Ausreißer-Punkte

In reellen Daten ist das Feature, welches man fair auf die Cluster verteilen will, meist nicht perfekt 1 : 1 verteilt. Damit man trotzdem ein faires Clustering durchführen kann, ist es möglich einige Punkte als Ausreißer zu bestimmen. Diese Ausreißer werden dann nicht mehr beim Clustering berücksichtigt und werden dementsprechend nicht einem Cluster zugewiesen.

In einem Verhältniss von $n : n + x$ Punkten werden n einzelne 1 : 1 Fairletts gebildet und x Ausreißer definiert.

2.5.2 Red-Clustering

Als erste Art des Clusterings betrachten wir das Red-Clustering. Der hier verwendete Algorithmus ist eine Abwandlung des Algorithmus, welcher von Chierichetti et al. [Chi+18] vorgestellt wurde für 1:1 Clustering. Hier wurde er erweitert, um mit der Berücksichtigung von Ausreißern arbeiten zu können.

2.5.2.1 Bilden der Red-Cluster Fairletts 2

Erst gewährleisten wir, dass das kritische Feature, welches seltener, also nur n mal vorkommt, das rote Feature wird. Das gilt ohne Beschränkung der Allgemeinheit, da wir die Farbe in linearer Laufzeit einfach tauschen können.

Anschließend bauen wir einen bipatiten Graphen auf, in dem die roten $R \in P$ und blauen $B \in P$ Punkte jeweils in eine Partition geteilt sind $R \cap B = \emptyset$, $R \cup B = P$ und $|R| \leq |B|$.

Als nächstes berechnen wir ein Matching mit einem Flussnetzwerk, um die Fairletts zu bestimmen. Dazu fügen wir Quelle s und Senke t in das Netzwerk hinzu und ergänzen Kanten zwischen Quelle und den roten Knoten bzw. Kanten zwischen blauen Knoten und Senke, welche alle die Kapazität

1 haben. Nun brauchen wir noch die Kanten zwischen den roten und den blauen Punkten. Dazu „raten“ wir ein r_{test} und fügen alle Kanten zwischen $a \in R$ und $b \in B$ ein, für welche gilt $dist(a, b) \leq r_{test}$. Dabei wählen wir das r_{test} aus der Liste von allen möglichen paarweisen Distanzen $r_{test} \in \{dist(a, b) | a \in R, b \in B\}$.

In diesem Netzwerk wird nun der maximale Fluss berechnet und wenn der Flusswert f gleich der Anzahl an roten Punkten ist $f = |R|$, wissen wir, dass alle roten Punkte gematcht wurden. Diese Berechnung führen wir so lange mit verschiedenen $r_{test} \in \{dist(a, b)\}$, bis wir das kleinste r_{test} gefunden haben, für die das Matching erfolgreich war ($f = |R|$).

Um die Anzahl der Vergleiche zu verringern sortieren wir die Menge $\{dist(a, b) | a \in R, b \in B\}$ aufsteigend und suchen r_{opt} über die Binäre-Suche. Dadurch haben wir statt der $\mathcal{O}(n^2)$ möglichen Vergleiche nur noch $\mathcal{O}(\log(n^2)) = \mathcal{O}(2 * \log(n)) = \mathcal{O}(\log(n))$.

Um jetzt die Fairletts einander zuzuweisen, erzeugen wir nochmal das Flussnetzwerk mit den Kanten $e_i = (a, b)$ für $\{dist(a, b) \leq r_{opt} | a \in R, b \in B\}$. Dann betrachten wir die Kanten zwischen den Partitionen R und B und sobald ein Fluss von 1 über die Kante e_i fließt, weisen wir a und b mit $e_i = (a, b)$ die selbe FairlettID zu, was sie zu einem Fairlett macht.

2.5.2.2 Finden der Ausreißer

Um die Ausreißer zu finden, müssen wir noch alle blauen Punkte suchen, welche nicht gematcht wurden. Für jeden blauen Punkt b welcher nicht gematcht wurde, gilt, dass der Fluss über die Kante $e_{t_i} = (b, t)$ 0 ist. Also gehen wir durch alle Senke-Kanten $e_{t_i} = (b, t)$ für alle $i \in [0, |B|]$ und wenn der Flusswert dieser Kante $f(e_{t_i}) = 0$ ist, dann markieren wir b als Ausreißer. Dieser Punkt wird bei der Zuweisung zu den Zentren ausgelassen

2.5.2.3 Berechnen der roten Zentren 3

Jetzt clustern wir lediglich die roten Punkten mithilfe des Algorithmus von Gonzalez 1. Dabei können wir sicher sein, dass dabei kein Ausreißer als Zentrum gewählt wird, da aufgrund der Bedingung am Anfang $|R| \leq |B|$ kein roter Punkt ein Ausreißer ist.

Nach diesem Schritt sind alle roten Punkten einem Cluster zugewiesen.

Algorithm 2 Fairlett-Finder Red Clustering

```

1: function RED-CLUSTER-FAIRLETT( $P$ ,  $\text{dist}$ )
2:    $R := \{x \mid x \in P \wedge x.\text{color} = \text{RED}\}$ 
3:    $B := \{x \mid x \in P \wedge x.\text{color} = \text{BLUE}\}$ 
4:    $R_{\text{pot}} = \{r = \text{dist}(a, b) \mid a \in R \mid b \in B\}$ 
5:    $G :=$  Graph für Flussnetzwerk
6:
7:   for all  $r_{\text{test}} \in R_{\text{pot}}$  do ▷ verwende Binäre Suche
8:     Füge in  $G$  alle Punkte aus  $R$  ein
9:     Füge in  $G$  alle Punkte aus  $B$  ein
10:    Füge in  $G$  Quelle  $s$  und Senke  $t$  ein
11:    Alle Kanten  $(s, a)$  und  $(b, t)$  mit Kapazität 1 einfügen
12:    for all  $\{(a, b) \mid a \in R \mid b \in B\}$  do
13:      if  $\text{dist}(a, b) \leq r_{\text{test}}$  then
14:        Füge Kante  $(a, b)$  in  $G$  mit Kapazität 1 ein
15:      end if
16:    end for
17:    Berechne Maximalen Fluss  $f$  von  $G$ 
18:    Vergleiche mit  $f \geq |R|$  für Binäre-Suche
19:  end for
20:
21:  Baue  $G$  neu auf mit  $r_{\text{opt}}$  wie in der oberen Schleife auf
22:  Berechne den maximalen Fluss  $f$  auf  $G$ 
23:  for all  $e_{\text{main}_i} \in \{(a, b) \mid a \in R \mid b \in B\}$  do ▷ markieren der Fairletts
24:    if  $f(e_{\text{main}_i}) = 1$  then
25:      Weise  $e.a$  und  $e.b$  dem gleichen Fairlett zu
26:    end if
27:  end for
28:
29:  for all  $e_t \in \{(b, t) \mid b \in B\}$  do ▷ Markieren der Ausreißer
30:    if  $f(e_t) = 0$  then
31:      markiere  $b$  als Ausreißer
32:    end if
33:  end for
34: end function

```

2.5.2.4 Zuweisen der blauen Partner

Zum Schluss gehen wir durch alle blauen Punkte durch und weisen sie demselben Cluster zu, zu welchem ihr roter Fairlett-Partner zugewiesen wurde oder überspringen sie, falls sie Ausreißer sind.

Algorithm 3 Red Clustering Algorithmus

```

1: function RED-CLUSTERING( $P$ ,  $dist$ ,  $k$ )
2:   Berechne und markiere Fairletts mit RED-CLUSTER-FAIRLETT( $P$ ,  $dist$ )
3:    $R := \{x | x \in P \wedge x.color = RED\}$ 
4:   Berechne Clustering von  $R$  mit GONZALEZ( $R$ ,  $dist$ ,  $k$ )
5:    $B := \{x | x \in P \wedge x.color = BLUE\}$ 
6:   for all  $b \in B$  do
7:     if  $b$  ist kein Ausreißer then
8:       weise  $b$  dem selben Cluster zu wie sein Fairlett Partner
9:     end if
10:  end for
11: end function

```

2.5.3 Fast-Anchor-Clustering

Als zweiten Clustering-Algorithmus betrachten wir den Fast-Anchor-Algorithmus [Fas23]. Dieser arbeitet ähnlich wie der Red-Clustering Algorithmus 2.5.2, bildet aber die Fairletts nach einer grundlegend anderen Bedingung.

2.5.3.1 Bilden der Anchor-Fairletts 4

Wie im Red-Clustering Algorithmus 2.5.2 bauen wir ein Flussnetzwerk zum Finden der Fairletts auf. Unterschiedlich ist nur die Bedingung für das Hinzufügen der Kanten zwischen $a \in R$ und $b \in B$. Man testet für alle Paare $(a, b) \in \{a \in R | b \in B\}$ jeden Punkt $\alpha \in P$, ob α als Anker in Frage kommt. Dabei wird der Anker so gewählt, dass $\max(dist(\alpha, a), dist(\alpha, b))$ minimal ist.

Diese Anker werden abgespeichert, denn durch sie bestimmen wir später, welchem Cluster wir das Fairlett zuweisen.

Beim Hinzufügen der Kanten zwischen $a \in R$ und $b \in B$ prüfen wir jetzt nicht mehr auf die Bedingung $dist(a, b) \leq r_{test}$, sondern betrachten den Anker, also $\max(dist(\alpha, a), dist(\alpha, b)) \leq r_{test}$ und

fügen dementsprechend Kanten mit Kapazität 1 ein.

Die Ausreißer werden genauso wie beim Red-Clustering bestimmt 2.5.2.2

2.5.3.2 Berechnen der Zentren

Wie bereits beim Red-Clustering 2.5.2 verwenden wir hier wieder den Algorithmus von Gonzalez, allerdings wird dieser erweitert. Beim normalen Gonzalez-Algorithmus 1 könnte es vorkommen, dass zwei Punkte eines Fairletts beide als Zentrum gewählt werden. Da sie aber ein Fairlett bilden, müssen sie beide dem gleichen Cluster angehören. Daher muss man immer, wenn man einen Punkt als Zentrum wählt, seinen Partner als potenzielles Zentrum entfernen. Wir benutzen hier also einen erweiterten Gonzalez-Algorithmus 5 um das eigentliche Clustering vorzunehmen. Diesmal clustern wir alle Punkte, welche keine Ausreißer sind, insbesondere auch blaue Punkte.

Die Zentren die in diesem Schritt berechnet werden, sind darüber hinaus immer gültig, da das Clustering ohne die Ausreißer berechnet wird.

2.5.3.3 Zuweisen der Punkte zu den Clustern

Zum Schluss müssen noch alle Punkte, welche keine Ausreißer, sind einem Cluster hinzugefügt werden. Dazu geht man alle Punkte $p \in P$ durch und betrachtet ihren Anker α_p . Man weist dann jeden Punkt p dem Zentrum $c \in Z$ zu, wobei Z die Menge der von Gonzalez 5 berechneten Zentren ist, wenn der Anker α_p diesem Zentrum am nächsten ist. Also $p.cluster = c.cluster$ wenn für c gilt $\min(dist(\alpha_p, c))$ für alle $c \in Z$.

2.5.3.4 Center-Aware Zuweisung

Da wir die Punkte dem Zentrum zuweisen, welches am nächsten am Ankerpunkt α ist, könnte es sein, dass ein Zentrum nicht Teil seines eigenen Clusters ist. Nehmen wir an, ein Punkt $c \in Z$ wird als Zentrum gewählt, dessen Anker α ist. Zusätzlich gibt es noch ein weiteres Zentrum $c_2 \in Z$ und es gilt $dist(c, \alpha) > dist(c_2, \alpha)$. Dann würde bei der Zuweisung in 6 der Punkt c dem Cluster von Punkt c_2 zugewiesen werden. Um das zu verhindern werden alle Zentren ihrem eigenen Cluster zugewiesen. Das gleiche machen wir mit den Fairlett-Partnern der Zentren.

Algorithm 4 Farilett-Finder Fast-Anchor Clustering

```

1: function FAST-ANCHOR-FAIRLETT( $P$ ,  $\text{dist}$ )
2:    $R := \{x | x \in P \wedge x.\text{color} = \text{RED}\}$ 
3:    $B := \{x | x \in P \wedge x.\text{color} = \text{BLUE}\}$ 
4:    $R_{\text{pot}} = \{r = \text{dist}(a, b) | a \in R | b \in B\}$ 
5:    $G :=$  Graph für Flussnetzwerk
6:    $A :=$  Ankermatrix  $|R| \times |B|$ 
7:
8:   for all  $(a, b) | a \in R | b \in B$  do                                     ▷ Ankerberechnung
9:     for all  $\alpha \in P$  do
10:      if  $\max(\text{dist}(\alpha, a), \text{dist}(\alpha, b)) \leq A[a, b]$  then
11:         $A[a, b] = \max(\text{dist}(\alpha, a), \text{dist}(\alpha, b))$ 
12:      end if
13:    end for
14:  end for
15:
16:  for all  $r \in R_{\text{pot}}$  do                                               ▷ verwende Binäre Suche
17:    Füge in  $G$  alle Punkte aus  $R$  ein
18:    Füge in  $G$  alle Punkte aus  $B$  ein
19:    Füge in  $G$  Quelle  $s$  und Senke  $t$  ein
20:    Alle Kanten  $(s, a)$  und  $(b, t)$  mit Kapazität 1 einfügen
21:    for all  $\{(a, b) | a \in R | b \in B\}$  do
22:      if  $\max(\text{dist}(A[a, b], a), \text{dist}(A[a, b], b)) \leq r$  then
23:        Füge Kante  $(a, b)$  in  $G$  mit Kapazität 1 ein
24:      end if
25:    end for
26:    Berechne Maximalen Fluss  $f$  von  $G$ 
27:    Vergleiche mit  $f \geq |R|$  für Binäre-Suche
28:  end for
29:
30:  Baue  $G$  neu auf mit  $r_{\text{opt}}$  wie in der oberen Schleife auf
31:  Berechne den maximalen Fluss  $f$  auf  $G$ 
32:  for all  $e_{\text{main}_i} \in \{(a, b) | a \in R | b \in B\}$  do                 ▷ markieren der Fairletts
33:    if  $f(e_{\text{main}_i}) = 1$  then
34:      Weise  $e.a$  und  $e.b$  dem gleichen Fairlett zu
35:    end if
36:  end for
37:  for all  $e_t \in \{(b, t) | b \in B\}$  do                               ▷ markieren der Ausreißer
38:    if  $f(e_t) = 0$  then
39:      Markiere  $b$  als Ausreißer
40:    end if
41:  end for
42: end function

```

Algorithm 5 Gonzalez-Algorithmus mit Fairletts

```

1: function GONZALEZ-FAIRLETT-AWARE( $P$ ,  $\text{dist}$ ,  $k$ )
2:    $Z :=$  leere Zentrenmenge
3:    $d_{\min} :=$  Distanz zu nächstem Zentrum
4:    $z_0$  beliebig aus  $P$ 
5:    $z_0.\text{cluster} = 0$ 
6:    $z_0.\text{partner.cluster} = 0$  ▷ Fairlett-Awareness
7:   Entferne  $z_0.\text{partner}$  als potenzielles Zentrum aus  $P$ 
8:
9:   for all  $x \in P$  do
10:      $d_{\min} = \text{dist}(z_0, x)$ 
11:      $x.\text{cluster} = 0$ 
12:   end for
13:
14:   for  $i \in 2, \dots, k$  do
15:      $z_i = P(\text{argmax}(d_{\min}))$ 
16:      $z_i.\text{cluster} = i$ 
17:      $z_i.\text{partner.cluster} = i$  ▷ Fairlett-Awareness
18:     Entferne  $z_i.\text{partner}$  als potenzielles Zentrum aus  $P$ 
19:     for  $x \in P \setminus Z$  do
20:       if  $\text{dist}(z_i, x) < d_{\min}[x]$  then
21:          $d_{\min}[x] = \text{dist}(z_i, x)$ 
22:          $x.\text{cluster} = i$ 
23:       end if
24:     end for
25:   end for
26:    $r_{\max} = \max(d_{\min})$ 
27:   return  $(Z, P, r_{\max})$  ▷ in  $P$  wurden Punkte aktualisiert
28: end function

```

Algorithm 6 Fast-Anchor Clustering Algorithmus

```

1: function FAST-ANCHOR-CLUSTERING( $P$ ,  $\text{dist}$ ,  $k$ )
2:   Berechne und markiere Fairletts mit FAST-ANCHOR-FAIRLETT( $P$ ,  $\text{dist}$ )
3:    $F := \{x | x \in P | x \text{ ist kein Ausreißer}\}$ 
4:   Berechne Clustering von  $F$  mit GONZALEZ-FAIRLETT-AWARE( $R$ ,  $\text{dist}$ ,  $k$ )
5:    $\rightarrow$  gibt Zentrenmenge  $Z$  zurück
6:
7:   for all  $p \in P$  enumerate  $i$  do            $\triangleright$  Zentren und Partner werden sich selbst zugewiesen
8:     if  $p$  ist Zentrum then
9:        $p.\text{cluster} = i$ 
10:       $p.\text{partner.cluster} = i$ 
11:    end if
12:  end for
13:
14:   $U := \{x | x \in P | x \text{ ist kein Zentrum, kein Ausreißer und kein Partner eines Zentrums}\}$ 
15:  for all  $p \in U$  do
16:     $\alpha := p.\text{anchor}$ 
17:    suche  $c \in Z$  sodass  $\text{dist}(\alpha, c)$  minimal
18:     $p.\text{cluster} = c.\text{cluster}$ 
19:  end for
20: end function

```

Dies kann in Sonderfällen aber zu einer Verschlechterung des maximalen Radius führen, obwohl man mehr Cluster zulässt. Dazu mehr im Kapitel Auswertung 5.1.1

Kapitel 3

Aufbereiten der Daten

Wir vergleichen unsere Algorithmen mit einem Ansatz von Chierichetti et al. [Chi+18], in welchem ebenfalls probiert wird, Fair-k-Center zu lösen. Um die experimentelle Güte zu vergleichen haben wir den selben Datensatz.

3.1 Ursprungsdaten

Chierichetti et al. [Chi+18] verwenden drei unterschiedliche Datensätze. Diese sind:

- Daten eines US-Zensus von 1994 [BK96]
- Daten von Telefongesprächen einer Bank [MRC12]
- Ein Diabetes-Datensatz, wobei der in [Chi+18] verlinkte Datensatz [Kah] nicht der richtige Datensatz sein kann (Der Datensatz enthält andere Felder als die im Paper genannten). Nachforschungen haben gezeigt, dass Chierichetti et al. in Wirklichkeit einen anderen Diabetes-Datensatz [Clo+14] benutzt haben.

3.1.1 Problem der Datendokumentation

Ein Problem der Daten ist, dass sie einerseits nicht in einem normalisierten CSV Format vorhanden sind und andererseits Chierichetti et al. [Chi+18] Subsamples benutzen, jedoch nicht angeben, welche Datenpunkte genommen und welche Features der Daten verwendet werden. Bei Letzterem werden für jeden Datensatz einige Features genannt. Diese wir hier verwenden. Die folgenden Experimente verwenden daher ein eigenes Subsampling, welches im Folgenden beschrieben wird.

3.2 Cleaning-Pipeline

Um die Daten verwendbar zu machen, brauchen wir einige Schritte. Dafür verwenden wir ein Python-Skript, welches einerseits automatisiert, andererseits deterministisch und konfigurierbar die Daten aufbereitet. Die Daten werden nach der Normalisierung, der Reinigung und dem Zerteilen jeweils abgespeichert, damit man ggf. mit diesen Daten weiterarbeiten kann, ohne sie noch einmal neu erzeugen zu müssen.

3.2.1 Normalisierung

Als Erstes werden die Daten in ein einheitliches CSV-Format übertragen. Das ist zwar bei dem Diabetes-Datensatz [Clo+14] nicht notwendig, bei den anderen zwei anderen Datensätzen [BK96] [MRC12] müssen Zeichen ersetzt werden und die Zensus-Daten werden um Überschriften ergänzt.

3.2.2 Reinigen und Filtern

Anschließend werden die Daten für unser Clustering weiterverarbeitet. Dazu werden alle Spalten entfernt, die wir nicht brauchen, sodass jeder Datensatz nur aus einigen numerischen Werten und dem Critical-Feature besteht. Die gewählten Spalten sind der Tabelle 3.1 zu entnehmen. Gleichzeitig wird auch eine Grundanalyse der Critical-Features (Verhältnisse und Mergen von Kategorien, welche das gleiche bedeuten) durchgeführt. Das Critical-Feature steht immer in der ersten Spalte. Ganz zum Schluss werden alle Daten auf ein Intervall von $[-1, 1]$ gemappt, indem alle

Feature jeweils durch den Betrag des maximalen Wertes geteilt werden. So haben Daten mit unterschiedlichen Skalierungen nicht unterschiedlichen große Auswirkung auf das Clustering bzw. die berechneten Distanzen.

3.2.3 Subsampling

Da es um faires Clustering geht, ist die Güte der Algorithmen stark abhängig von der Verteilung der Ursprungsdaten. Um zu vermeiden, ein sehr ungünstiges Subsample zu bekommen, werden alle Daten in 20 disjunkte Subsamples zerteilt, welche die selbe Größe haben wie in [Chi+18]. In der Analyse werden alle Subsamples getestet und der Durchschnitt sowohl über die Laufzeit als auch die Cluster-Größe gebildet.

3.2.4 Format der Datensätze

Die folgende Tabelle dokumentiert die verwendeten Datensätze. Von jedem Datensatz wurden jeweils 20 disjunkte Subsamples erzeugt.

Tabelle 3.1: Datenformat

Datensatz	Critical-Feature	Verhältnis des Critical-Feature	Verwendete numerische Features	Sub-sample-Größe
Zensusdaten [BK96]	sex (male = 0; female = 1)	Male 21790 / Female 10771 ≈ 2.02	age, fnlwgt, education-num, capital-gain, hours-perweek	600
Bankdaten [MRC12]	bin-martial (selbsterzeugt aus Spalte "marital") (single = 0; divorced = 0; married = 1)	Es gibt single, married und divorced. Zähle single und divorced zusammen. Verhältnis: 27214 married / 17997 not-married ≈ 1.51	age, balance, duration of call	1000
Diabetesdaten [Clo+14]	sex (male = 0; female = 1)	Verhältnis Male 47055 / Female 54708 ≈ 0.86	age, time-in-hostpial	1000

Kapitel 4

Experimente

Um die oben genannten Algorithmen 2 experimentell zu testen, nutzen wir eine Test-Pipeline, mithilfe derer Algorithmen automatisiert auf den Subsamples für verschiedene Clustermengen k ausgeführt werden, nachdem die Daten aufbereitet wurden.

4.1 Implementierung der Algorithmen

Die Algorithmen sind alle in C++ implementiert. Für die Flussalgorithmen 2.2.2 wird die Open-Source Bibliothek Lemon Graph Library [Lem] verwendet. Gesteuert werden sie durch ein Hauptskript `ClusteringMain.cpp`, welches über die Kommandozeile bedient werden kann. In der `ClusteringTest.cpp` sind einige Test-Methoden, welche beim Entwickeln der Algorithmen verwendet wurden, um die einzelnen Schritte zu überprüfen.

Das Program lässt sich in zwei Modi kompilieren (setzen des Flags `-DPROCESS_BAR`). Es lässt sich so entscheiden, ob eine Fortschrittsanzeige auf der Standardausgabe angezeigt wird. Dies wird für die automatischen Tests deaktiviert, aus Gründen der Übersichtlichkeit. Bei der tatsächlichen Anwendung zur Datenverarbeitung mit größeren Daten ist diese Statusausgabe von Vorteil.

4.2 Automatisierte Analyse

Wie zuvor genannt, sollen die Algorithmen auf einigen Subsamples und mit verschiedener Clusteranzahl k durchgeführt werden. Um einen Algorithmus auf einem Subsample für verschiedene k durchzuführen, verwenden wir ein Bash-Skript `AlgFeeder.sh`. Dieses führt sukzessiv für ein bestimmtes Sample und einen Algorithmus jedes Clustering von 1 bis k Cluster durch und speichert die Ergebnisse in einem Ordner.

Um die verschiedenen Subsamples zu testen, wird im Pythonskript (`AlgorithmDataCruncher.py`) mithilfe von Subprozessen eine Shell geöffnet und für jedes Sample das Bash-Skript `AlgFeeder.sh` aufgerufen. Nach der Berechnung sind die geclusterten Daten in CSV-Dateien gespeichert und werden wieder von einem anderen Pythonskript (`DataAnalyzer.py`) eingelesen, welches dann die Ergebnisse analysiert und die Graphen erstellt. Alles zusammen wird in `AutoAnalyzer.py` automatisch nacheinander ausgeführt. Die Unterteilung wurde deshalb gemacht, um die Analyse mehrfach anzupassen zu können ohne jedes Mal das ganze Clustering neu berechnen zu müssen.

Die Verwendung des Moduls `subprocess` und des Bash-Skriptes führen zu einer Plattformabhängigkeit auf GNU/Linux für das automatisierte Testen. Die C++ Implementierung ist jedoch plattformunabhängig.

Es werden auf allen Samples jeweils drei Algorithmen ausgeführt (Gonzalez 1, Red-CLustering 2.5.2 und Fast-Anchor-Clustering 2.5.3)

4.3 Testumgebung

Im Folgenden werden die technischen Daten der Umgebung in welcher die Tests durchgeführt wurden aufgeführt:

- OS: Ubuntu 20.04 LTS
- Prozessor AMD Ryzen 7 5800x
- RAM: 16 GB

Versionierung von Python und den Modulen:

- Python: 3.11.8
- Matplotlib: 3.8.0
- Numpy: 1.26.4
- Pandas: 2.2.1
- pip: 23.3.1

Compiler für C++ Implementierung: g++ 9.4.

4.4 Graphen

4.4.1 Güte-Analyse

Um die Algorithmen zu vergleichen, betrachten wir ihre Güte (also die Größe der produzierten Cluster) auf den Testdaten mit verschiedener Anzahl an Clustern. Die Graphen 4.1 zeigen auf der x-Achse wie viele Cluster gebildet wurden und auf der y-Achse wie groß die Kosten der Cluster sind. Dabei wird für jedes k der Mittelwert der Kosten aller Subsamples des entsprechenden Datensatzes gebildet. Der helle Bereich um den Mittelwert gibt die Breite der Varianz der Ergebnisse an. In unserem Fall betrachten wir das k-center Problem. Daher sind unsere betrachteten Kosten der maximale Radius, den ein Cluster in der berechneten Lösung besitzt. Zusätzlich wird noch der Fairlett-Radius angegeben, mit dem größten Abstand zwischen zwei Fairlett-Partner bzw. im Falle des Fast-Anchor Clusterings 2.5.3 wie weit ein Fairlett-Punkt von seinem Anker maximal entfernt ist.

Die einzelnen Graphen sind aufgeteilt auf die verschiedenen Datensätze.

4.4.1.1 Randfall Güteverschlechterung

Für Fast-Anchor-Clustering 2.5.3 wird noch ein zusätzlicher Graph erzeugt 4.2. Auf diesem sind nicht nur die Durchschnittskosten zu sehen, sondern auch die maximalen und die minimalen Kosten der jeweiligen k .

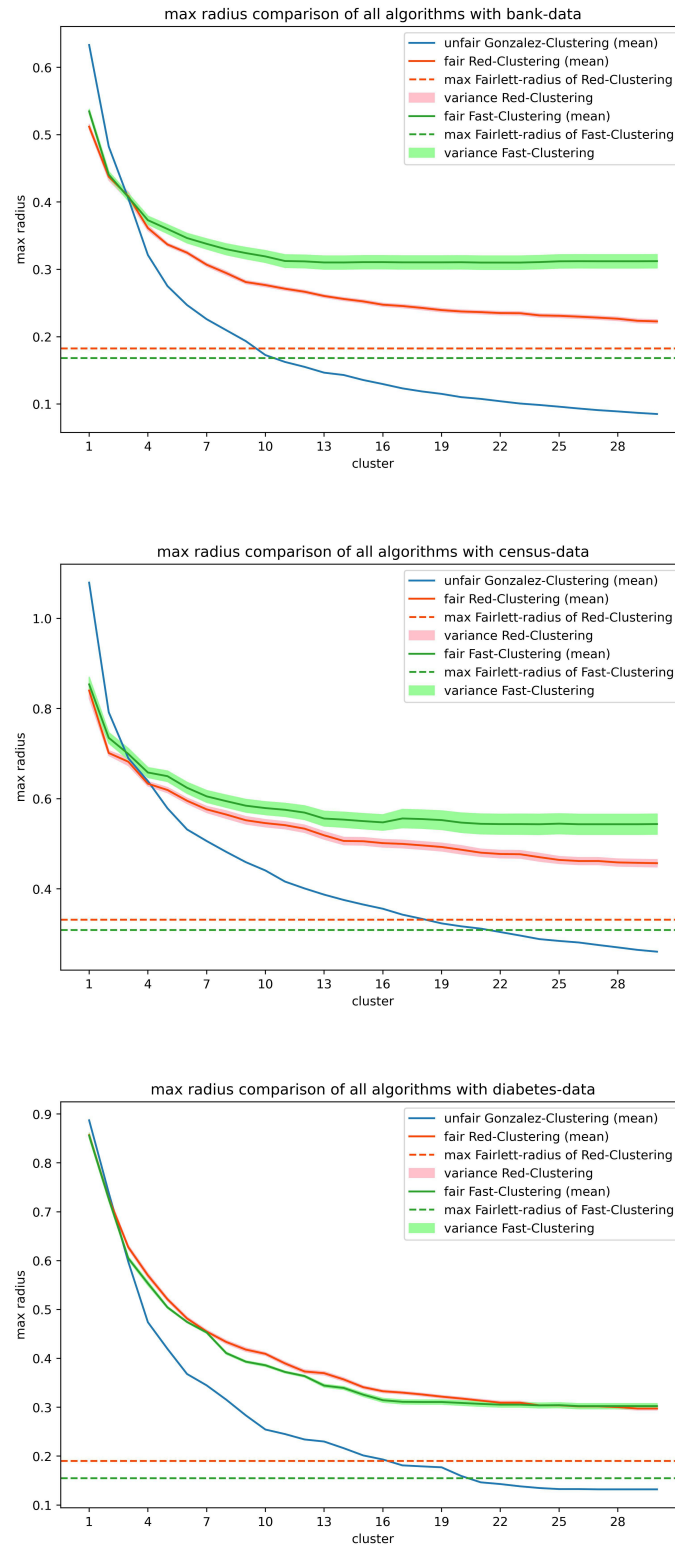


Abbildung 4.1: Zusammengefasste Kosten der Cluster bis zu 30 Cluster

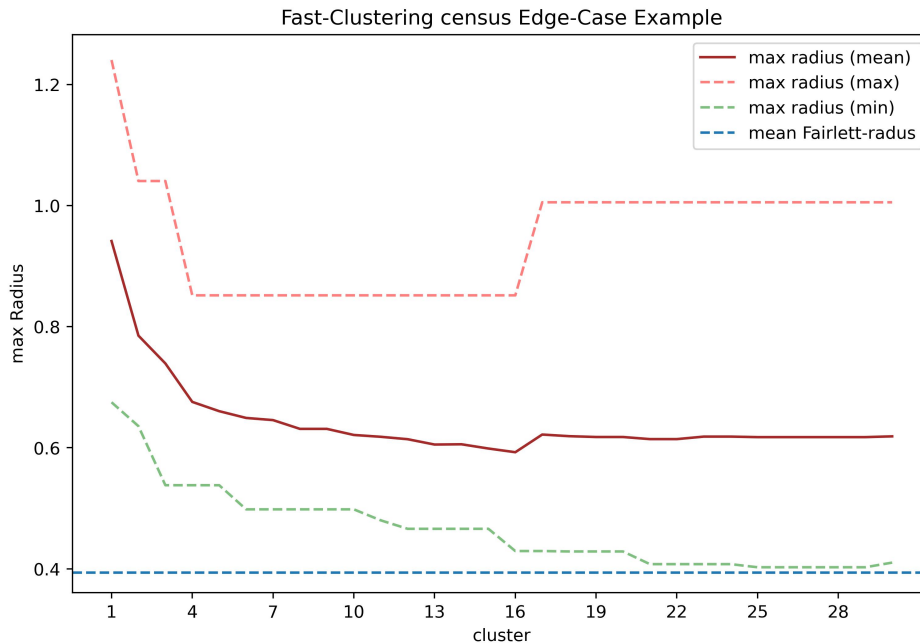


Abbildung 4.2: Kosten der Cluster mit Median und Min/Max des Zensusdatensatzes mit den ersten 10 Subsamples

4.4.1.2 Min-Cost-Test

Wir haben getestet, ob sich die Ergebnisse verbessern lassen, wenn man beim Fast-Anchor-Clustering 2.5.3 statt eines Max-Flows einen Min-Cost-Flow 2.2.3 benutzt und die Unterschiede (auf einem kleineren Teildatensatz) berechnet 4.3. Links wird die Analyse mit einem Max-Flow 2.2.2 und rechts die Analyse mit einem Min-Cost-Flow 2.2.3 dargestellt.

4.4.2 Laufzeit-Analyse

Um die Laufzeit der Algorithmen zu analysieren, verwenden wir in Python aus dem Modul `time` die Funktion `time.process_time_ns()`, welche die Prozessor-Zeit misst, in welcher der Aufruf des Bash-Skriptes rechnet. Die einzelnen Graphen zeigen die Laufzeit eines Algorithmus, wobei die verschiedenen Datensätze verglichen werden. Zusätzlich ist der Mittelwert der Laufzeit für jeden Datensatz in der Legende mitangegeben.

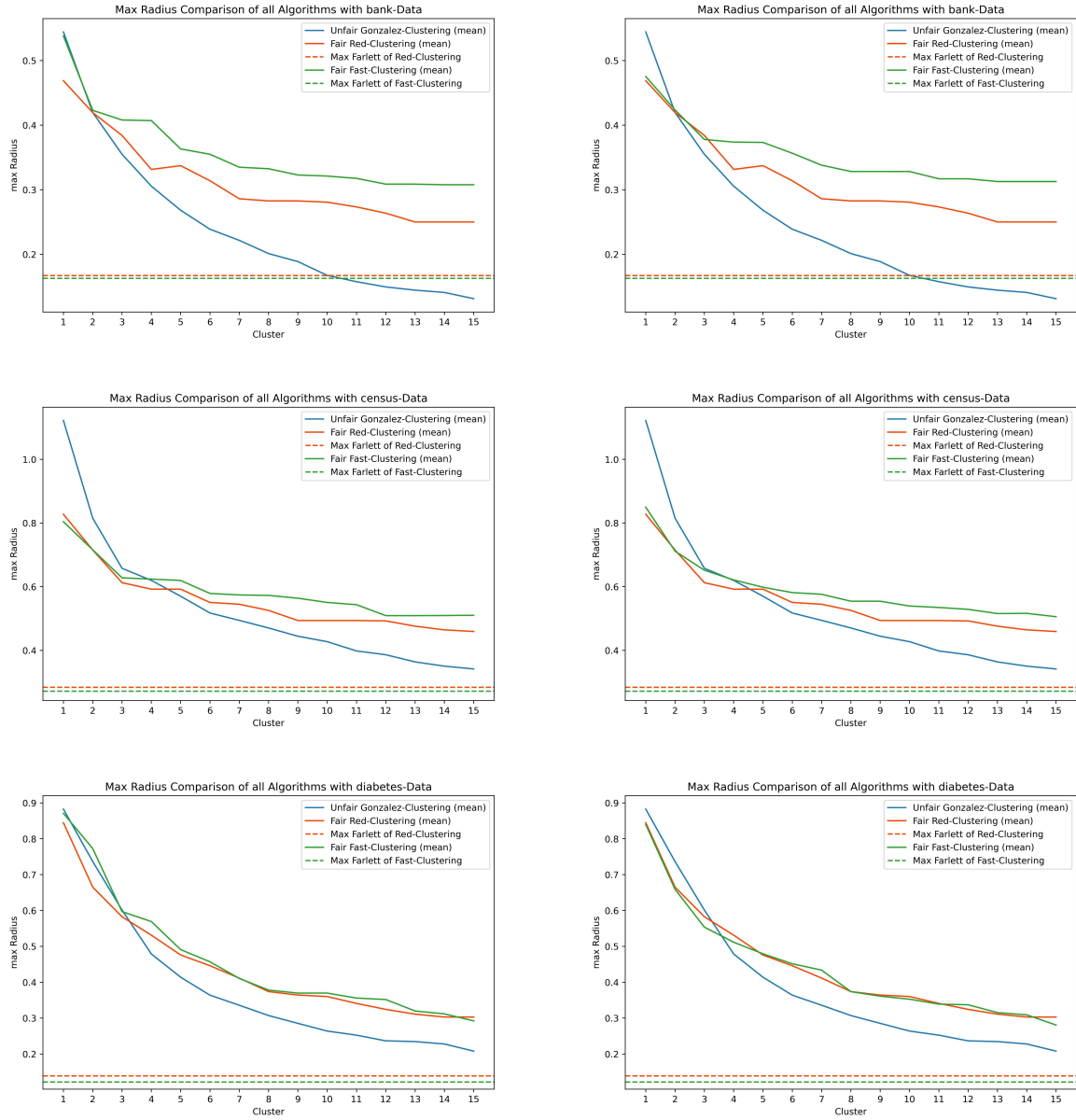


Abbildung 4.3: Unterschied in den Radiuskosten der Cluster zwischen Fairlettbildung mit einfachen Max-Flow und Min-Cost-Flow bei Fast-Anchor-Clustering

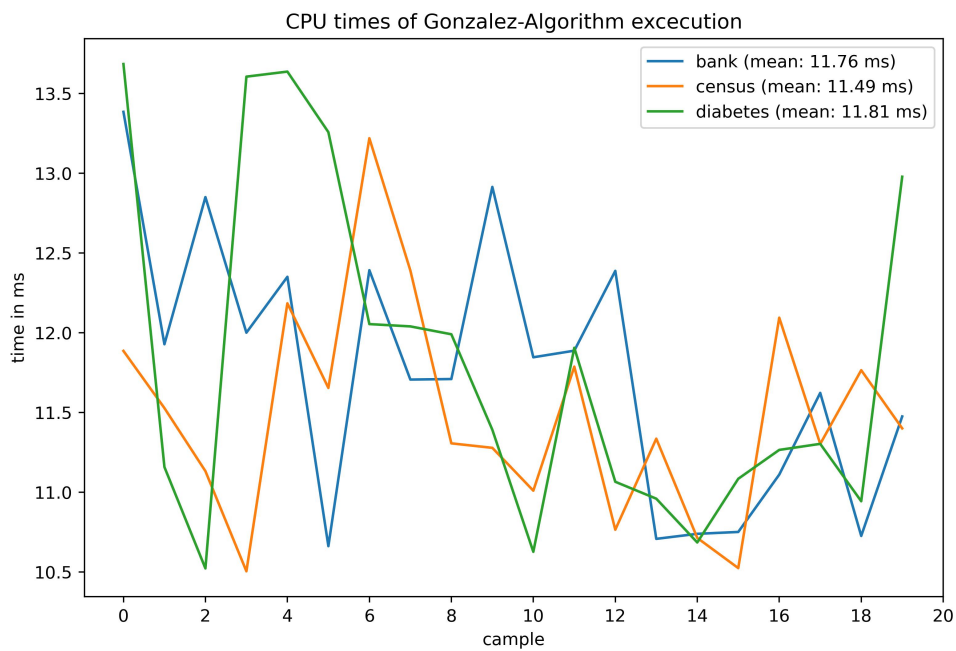


Abbildung 4.4: Laufzeit des Gonzalez-Algorithmus



Abbildung 4.5: Laufzeit des Red-Clustering-Algorithmus

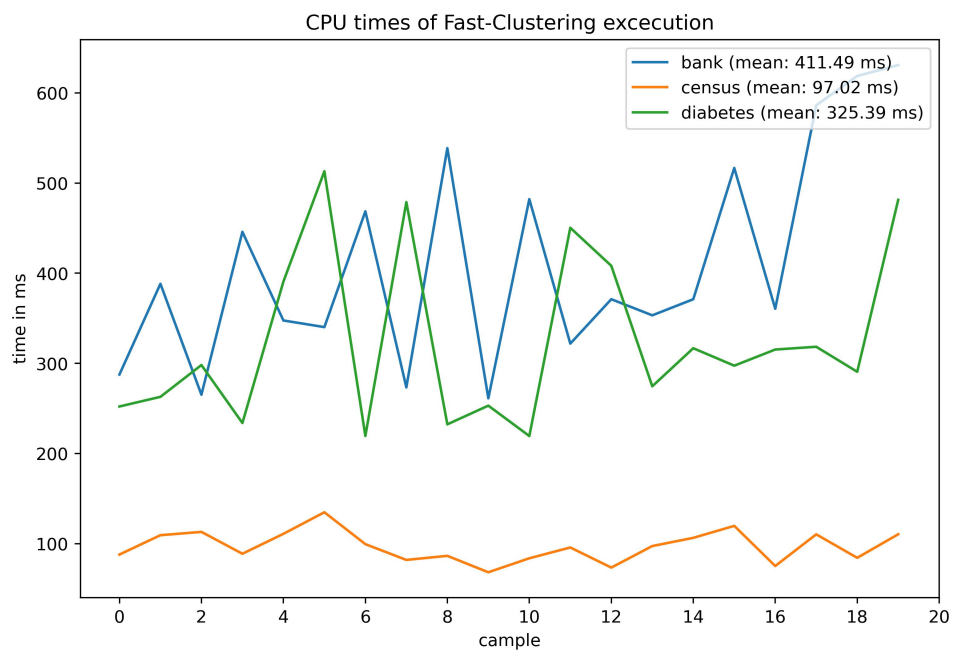


Abbildung 4.6: Laufzeit des Fast-Anchor-Algorithmus

Kapitel 5

Auswertung

5.1 Güte der Algorithmen

Wenn wir uns die Graphen 4.1 anschauen, dann sehen wir, dass alle Algorithmen mit steigendem k Cluster mit geringeren Kosten berechnen. Dabei handelt es sich allerdings um den Median über jeweils 20 Subsamples pro Datensatz. Auffällig ist, dass die Kurve für das Fast-Anchor-Clustering 2.5.3 nicht monoton fällt. Das ist in den Graphen mit Median 4.1 für den Datensatz der Zensusdaten [BK96] von $k = 16$ auf $k = 17$ zu erkennen. Besser noch im einzelnen Graphen 4.2, in welchem an der Maximal-Linie klar eine Steigerung zu sehen ist. Ein Einblick in die Daten zeigt, dass es beim Fast-Anchor-Clustering 2.5.3 vorkommen kann, dass ein neuer Punkt als Zentrum gewählt wird und dem neuen Cluster dann dieses Zentrum selbst und der Partnerknoten zugewiesen werden müssen. Das kann die Kosten erhöhen 5.1.1.

Es ist ebenfalls zu sehen, dass das Red-Clustering bei den Datensätzen der Bankdaten [MRC12] und der Zensusdaten [BK96] merklich bessere Cluster berechnet. Besser meint hier Cluster mit geringeren Kosten. Der Abstand der Kosten der Algorithmen ist größer als die Varianz beider Algorithmen und daher nicht auf natürliche Schwankungen zurückzuführen. Bei den Diabetes-Daten [Clo+14] ist das Fast-Anchor-Clustering besser, jedoch nur sehr gering im Gegensatz zu den anderen Datensätzen. Auffallend ist, dass der Abstand zwischen den Algorithmen mit der Anzahl an verwendeten Feldern korreliert. Diese Beobachtung wird in dieser Arbeit jedoch nicht weiter verfolgt.

Das gewählte r_{opt} für die Fairlettbildung ist in allen Fällen beim Fast-Anchor-Clustering besser. Das liegt vermutlich daran, dass die Fairletts, welche beim Red-Clustering gebildet werden 2, ebenso eine legitime Lösung für die Fairlettbildung beim Fast-Anchor-Clustering 4 sind.

5.1.1 Center-Aware Problem

Beim Fair-Anchor-Clustering 2.5.3 weisen wir explizit die Punkte, welche Zentren sind, automatisch ihren eigenen Clustern zu. Außerdem werden ihre Fairlett-Partner demselben Cluster zugewiesen. Approximativ wird die Güte dadurch nicht schlechter [Fas23] jedoch fällt bei den Experimenten auf, dass bei spezifischen Datensamples 4.2 der maximale Radius des Clusters größer wird, obwohl wir mehr Zentren zulassen.

Durch weitere Nachforschung stellte sich heraus, dass es tatsächlich Randfälle gibt, in welchen der maximale Radius wieder ansteigen kann, wenn man ein neues Cluster dazu nimmt.

5.1.1.1 Beispiel einer Radiussteigung

In Abbildung 5.1 und 5.2 sehen wir einen solchen Fall bei einer Steigerung von $k = 3$ auf $k = 4$. Voraussetzung dafür ist, dass ein Fairlettpaar $r \in P$ und $b \in P$ im Fall $k = 3$ Teil des Clusters c_0 ist. Im Beispielbild 5.1 ist zu sehen, dass r mit $\text{dist}(r, c_0)$ maßgeblich für den maximalen Radius ist. Dabei sind r und b dem Cluster von c_0 zugewiesen weil die Distanz des Ankers α zum Zentrum c_0 am kleinsten ist.

Im nächsten Schritt wird r als neues Zentrum (c_4) gewählt, weil es der am weitesten entfernte Punkt von allen anderen Zentren ist. Allerdings muss jetzt sowohl r selbst als auch sein Fairlettpartner b dem neuen Cluster c_4 zugewiesen werden. Dadurch erhöht sich der maximale Radius enorm auf $\text{dist}(r, b)$.

5.1.2 Tests mit Min-Cost-Flow

Um den Fast-Anchor-Algorithmus 2.5.3 etwas zu verbessern, wurde probiert, den Max-Flow 2.2.2, welcher bei der Fairlettbildung 2.5.3.1 verwendet wird, durch einen Min-Cost-Flow 2.2.3 zu ersetzen. Die Idee war, dass so Punkte gewählt werden die nicht nur nah am Anker sind, sondern auch möglichst dicht beieinander. Jedoch ist dem Graphen 4.3 zu entnehmen, dass die Verbesserungen sehr klein sind. In dieser Arbeit wurde deshalb der Min-Cost-Flow nicht verwendet.

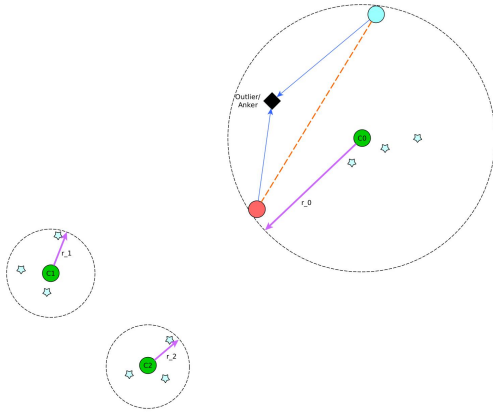


Abbildung 5.1: Fast-Anchor-Clustering
Edge-Case mit $k = 3$

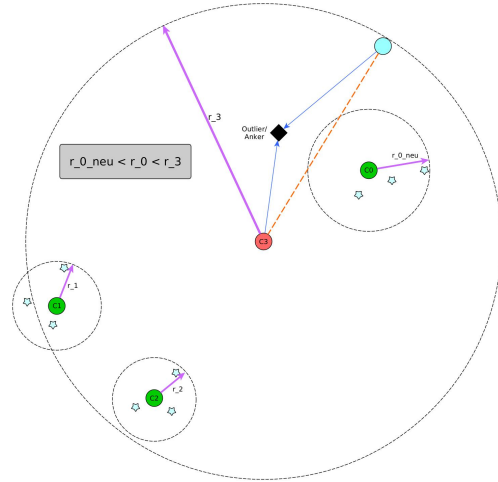


Abbildung 5.2: Fast-Anchor-Clustering
Edge-Case mit $k = 4$

5.2 Laufzeitanalyse

Vergleichen wir die Laufzeit des Red-Clusterings 4.5 mit der des Fast-Anchor-Clusterings 4.6 stellen wir fest, dass die Laufzeit des Fast-Anchor-Clusterings mehr als doppelt, im Falle der Diabetest-Daten [Clo+14] sogar fast das viermal so groß ist.

Es ist zu erwarten, dass der Fast-Anchor Algorithmus länger braucht, da er zusätzlich zum Finden der Fairletts auch die Anker berechnen muss 4. Diese Berechnung ist mit einer theoretischen Laufzeit von $\Theta(n^3)$ sehr teuer.

Es ist ebenfalls erwartbar, dass die Laufzeit von Gonzalez 4.4 um einiges schneller ist. Er hat nicht nur die bessere theoretische Laufzeit von $\mathcal{O}(n * k)$ [Gon85], sondern wird auch von den anderen Algorithmen als Unteroutine benutzt.

5.3 Mögliche Verbesserungen

Folgende Verbesserungen könnten noch durchgeführt werden, um einerseits bessere Algorithmen und andererseits eine bessere Analyse zu erhalten.

5.3.1 Laufzeit

Die Implementierung der Algorithmen ist in der Laufzeit zwar approximativ optimal, jedoch könnten im Code noch einige Mikroverbesserungen durchgeführt werden. Darunter fallen einige Berechnungen, welche mehrfach durchgeführt werden. Die paarweiße Distanz zwischen den Punkten werden an verschiedenen Stellen gebraucht und man könnte die Werten einmalig berechnen und dann nur noch weitergeben. Dies wurde bisher zum Wohle der Übersichtlichkeit nicht implementiert und um die Richtigkeit der Algorithmen zu gewährleisten.

Wenn wir einen Radius beim Suchen der Fairletts „überprüfen“, wird jedesmal ein neues Flussnetzwerk erzeugt. Alternativ wäre es möglich, das bestehende Flussnetzwerk zu behalten und stattdessen die Kapazitäten nur zu ändern. Insbesondere müssen nur die Kapazitäten der Kanten zwischen den roten und den blauen Punkten aktualisiert werden.

Um die Laufzeitmessung klarer zu halten, ist die Implementierung single-threaded. Dabei lassen sich einige Berechnungen parallelisieren. Darunter fällt die Ankerberechnung der Paare und das Filtern und Zuweisen der Punkte nach dem Clustering.

5.3.2 Kosten

Der Algorithmus von Gonzalez [Gon85] hat die Tendenz Cluster zu bilden, bei denen das Zentrum am Rand liegt. Es wäre möglich, in einem weiteren Schritt noch über jedes Cluster zu gehen und in diesem das optimale Zentrum zu suchen. Das ist in quadratischer Zeit möglich, da jeder Punkt nur einmal als potenzielles Zentrum gewählt wird und dann die Distanz zu allen anderen Punkten gemessen werden muss.

Ebenfalls könnte man die Fairlett-Suche des Fast-Anchor-Clusterings 4 noch verbessern. Bei der Wahl der Fairletts wird lediglich die Distanz zum Anker betrachtet, aber nicht der Abstand zwischen den Partnern. Wie in 5.1.1 gezeigt, führt das gelegentlich zu größeren Clustern. Alternativ könnte man bei der Fairlettbildung 2.5.3.1 einen Min-Cost-Flow 2.2.3 berechnen, wobei die Kosten der Distanz der Partner in einem Fairlett entsprechen. Da dies nur eine kleine Verbesserungen bringt 4.3 und die Original-Algorithmen [Fas23] nur mit einem einfachen Max-Flow 2.2.2 angegeben sind, wurde dies in der vorliegenden Arbeit nicht weiter berücksichtigt.

5.3.3 Analyse

Mit weiteren Datensätzen ließe sich zeigen, ob der Effekt der Radiussteigerung bei größerem k 5.1.1 wirklich regelmäßig auftritt. Zusätzlich könnte eine gewissere Aussage über die Korrelation zwischen Güteverschlechterung und Anzahl der verwendeten Felder getroffen werden. Dazu müsste man die Algorithmen vergleichen und die Anzahl der Felder variieren.

Um eine wahre Aussage über die Güte zu machen, bräuchte man zu den analysierten Datensätzen optimale Lösungen. Diese könnte man dann mit den Lösungen der Algorithmen vergleichen. Dadurch würde man eine genauere Aussage über die Güte der Algorithmen treffen können.

5.4 Fazit

In der Praxis ist der Fast-Anchor Algorithmus 2.5.3 nicht nur langsamer, sondern produziert auch in einigen Fällen eine schlechtere Lösung. Das ist überraschend, denn für den Red-Clustering Algorithmus 2.5.2 wurde eine Approximationsgarantie von $\mathcal{O}(n^4)$ gezeigt [Chi+18], während für das Fast-Anchor-Cluster eine Approximationsgarantie von $\mathcal{O}(n^3)$ gezeigt wurde [Fas23].

Ursachen für dieses Ergebniss sind möglicherweise die Daten, durch die Sonderfälle wie in 5.1.1 auftreten. Dieser Effekt wird noch verstärkt dadurch, dass die Fairletts beim Fast-Anchor-Clustering theoretisch fast doppelt so weit voneinander entfernt sein können als beim Red-Clustering. Das liegt daran, dass beim Bilden der Fairletts 4 nur der Abstand zum Anker α für beide Punkte (a, b) betrachtet wird. Für diesen gilt $dist(\alpha, a) + dist(\alpha, b) \leq dist(a, b)$, da wir eine Metrik verwenden und dementsprechend im schlimmsten Fall $dist(a, b) = dist(\alpha, a) + dist(\alpha, b) = 2 * dist(\alpha, a)$. Dadurch wird bei der Wahl von a als neues Zentrum der maximale Radius ggf. auf $dist(\alpha, a) + dist(\alpha, b) = 2 * dist(\alpha, a)$ verschlechtert, was doppelt so groß wie unser Fairlettradius ist.

Dabei sind wahrscheinlich beide Algorithmen in der Praxis deutlich besser als die Approximationsgarantie, welche sie versprechen. Dies wird in dieser Arbeit jedoch nicht überprüft, da man zur Überprüfung dieser These zu jedem Datensatz eine optimale Lösung haben müsste um das zu vergleichen.

Literatur

- [BK96] Barry Becker und Ronny Kohavi. *Adult*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5XW20>. 1996.
- [Chi+18] Flavio Chierichetti, Ravi Kumar, Silvio Lattanzi und Sergei Vassilvitskii. *Fair Clustering Through Fairlets*. 2018. arXiv: 1802.05733 [cs.LG].
- [Clo+14] John Clore, Krzysztof Cios, Jon DeShazo und Beata Strack. *Diabetes 130-US hospitals for years 1999-2008*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5230J>. 2014.
- [Fas23] Irina Fast. „Fair k-Center Clustering mit Ausreißern“. Düsseldorf, NRW, Germany, Dez. 2023.
- [Gon85] Teofilo F. Gonzalez. „Clustering to minimize the maximum intercluster distance“. In: *Theoretical Computer Science* 38 (1985), S. 293–306. ISSN: 0304-3975. DOI: [https://doi.org/10.1016/0304-3975\(85\)90224-5](https://doi.org/10.1016/0304-3975(85)90224-5). URL: <https://www.sciencedirect.com/science/article/pii/0304397585902245>.
- [Kah] Michael Kahn. *Diabetes*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5T59G>.
- [Kro] Carsten Krollmann. *Implemetation Bachelor-Arbeit-Clustering*. <https://gitlab.cs.uni-duesseldorf.de/krollmann/Bachelor-Arbeit-Clustering>.
- [Lem] A Jüttner, B Dezsö und P Kovács. *Lemon Library - open source graph library*. <https://lemon.cs.elte.hu/trac/lemon>.
- [Mac+67] James MacQueen u. a. „Some methods for classification and analysis of multivariate observations“. In: *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*. Bd. 1. 14. Oakland, CA, USA. 1967, S. 281–297.

- [MRC12] S. Moro, P. Rita und P. Cortez. *Bank Marketing*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5K306>. 2012.
- [Sch23a] Melanie Schmidt. „Graphenalgorithmen 1 - Vorlesungsskript“. 2023.
- [Sch23b] Melanie Schmidt.
„Kombinatorische Algorithmen für Clusteringprobleme - Vorlesungsskript“. 2023.

Ehrenwörtliche Erklärung

Hiermit versichere ich, die vorliegende Bachelorarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben. Alle Stellen, die aus den Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Düsseldorf, 07. Juni 2024

Carsten Krollmann